

PhishGuard

A Browser Extension for Real-Time, Locally Aware Phishing Detection

Cybersecurity Graduation Project — Full Technical Report

INSA Cybersecurity Training Program
Prepared by: Addisu
26 October 2026

Table of Contents

1. Introduction **3**

2. Problem Statement **3**

2.1 The problem and why it matters 3

2.2 The gap this project addresses 3

2.3 Scope: what is and is not addressed 4

3. Objectives **4**

4. Research and Analysis **4**

4.1 Technical causes: how phishing pages deceive 4

4.2 Existing solutions and their limits 5

4.3 Security implications and design consequences 5

5. System Architecture **6**

6. Implementation **7**

6.1 Scoring model 7

6.2 URL heuristics 7

6.3 Content heuristics 8

6.4 Local-context layer 9

6.5 Pre-click link scanning 9

6.6 Extension shell 9

6.7 Local server and input handling 10

7. Testing and Evaluation **10**

7.1 Unit, functional and browser tests 10

7.2 Evaluation method 11

7.3 Results 12

7.4 Example walkthroughs 13

8. Technical Demonstration **14**

9. Scope and Limitations **16**

10. Risks and Mitigations **17**

11. Development Timeline and Status **18**

12. Future Work **18**

13. Conclusion **19**

References **19**

1. Introduction

Phishing remains one of the most common and effective ways attackers steal credentials, financial information, and personal data. Rather than exploiting a flaw in software, phishing exploits human trust: the attacker impersonates a legitimate brand, government agency, or financial institution and persuades the victim to enter sensitive information on a fraudulent page. Because it targets people rather than systems, phishing keeps succeeding even as other classes of attack are hardened.

This report presents PhishGuard, a browser extension with a Python detection engine that analyzes the web pages and links a user encounters and warns them before they submit sensitive information to a suspicious site. Most phishing-detection tools are built around English-language content and international brands. PhishGuard adds a local-context layer aimed at threats relevant to Ethiopian users: impersonation of local banks, mobile money services and government portals, and scam wording in Amharic.

The report follows the structure of the graduation project brief: problem identification (Section 2), objectives (Section 3), research and analysis (Section 4), architecture and implementation (Sections 5–6), testing and evaluation (Section 7), technical demonstration (Section 8), and limitations, risks, timeline, future work and conclusion (Sections 9–13).

2. Problem Statement

2.1 The problem and why it matters

Phishing is large, persistent, and growing. The Anti-Phishing Working Group (APWG) recorded about 3.8 million phishing attacks in 2025, up slightly from 3.76 million in 2024, with social media and SaaS/webmail the most attacked sectors [1]. In the first quarter of 2026 it recorded 971,181 attacks, 13.8 percent more than the 853,244 of the previous quarter, and reported that scams were rising on every social media platform and that telecom brands were increasingly targeted [2].

Ethiopia is not exempt. INSA warned in early 2026 of a widespread WhatsApp campaign in which links disguised as election information or university notices were used to harvest personal data and take over accounts, often sent from the compromised accounts of trusted contacts [3]. Ethio telecom documents telebirr fraud carried out through fraudulent calls and SMS and fake banking-style apps, and urges customers never to share one-time codes [4]. A 2024 interview in Capital describes phishing in Ethiopian banking as mostly delivered by text message and phone call, using USSD short codes [5].

2.2 The gap this project addresses

Mainstream browser protection is strong against widely reported campaigns, but it is built on reputation: a URL must usually be known to be bad before it is blocked. Google reports that Chrome’s lists were historically refreshed every 30 minutes or so and that some phishing sites slip through that window by switching domains quickly or hiding from crawlers, which is why it introduced real-time checks [6][7]. Newly registered phishing domains, which are often used and abandoned within hours, are exactly the hardest case for any reputation-based system.

In addition, general tools are not tuned to local institutions. A page that imitates a regional bank or mobile money service, or that uses Amharic urgency wording, gives them no special signal.

PhishGuard treats this as the real-world problem: provide a lightweight, transparent, and locally aware layer of phishing protection that inspects what the user is actually looking at (URL structure, page content and the links on the page) and explains its reasoning.

2.3 Scope: what is and is not addressed

A large share of phishing in Ethiopia reaches victims by SMS, voice calls and USSD [4][5]. A browser extension cannot see any of these. PhishGuard addresses the part of the problem that ends in a web page: links shared through webmail and the web versions of social platforms and messengers, and the fraudulent pages themselves. SMS, voice and native-app channels are explicitly out of scope (Section 9).

3. Objectives

The project has one primary objective and several supporting objectives:

- Design and build a working browser extension that analyzes pages a user visits and flags likely phishing attempts in real time.
- Combine multiple independent signals (URL structure, domain characteristics, page content, language) rather than relying on one indicator, to reduce false positives and false negatives.
- Incorporate a local-context layer: a watchlist of commonly impersonated local institutions and recognition of common Amharic scam phrasing.
- Extend detection to links on any web page, so risky links can be flagged before the click.
- Evaluate the system on known phishing and legitimate pages and report measurable results, including failures.
- Document the design, limitations and recommendations for future work.

4. Research and Analysis

4.1 Technical causes: how phishing pages deceive

Phishing succeeds because several cheap techniques make a fraudulent page look legitimate for the few seconds in which a user decides to trust it. The techniques below drive the signals PhishGuard computes.

Technique	How it works	PhishGuard signal
Typosquatting	Registers a domain one or two characters away from a real brand (for example a swapped or dropped letter).	Edit distance to every watchlist domain (`typosquat`).
Homoglyphs	Swaps look-alike characters (`1` for `l`, `0` for `o`, `rn` for `m`, Cyrillic letters).	Normalizes confusable characters before comparing (`homoglyph`).

Technique	How it works	PhishGuard signal
Brand in a hostname	Puts a real brand name inside an unrelated domain such as `brand-secure-login.example`.	Brand keyword found in hostname tokens (`brand_in_domain`).
Obfuscated destinations	Raw IP addresses, `@` tricks, many subdomains and hyphens, punycode, link shorteners.	Individual URL signals (Section 6.2).
Cloned login pages	Copies a brand's title and logo, hosts it elsewhere, and posts the form to the attacker.	Brand/domain mismatch plus login form; external form action (Section 6.3).
Hidden form fields	Off-screen or invisible inputs that capture autofilled data.	`offscreen_fields`.
Urgency and prize language	Pressures the victim to act without checking (account suspended, you have won).	Scam-phrase matching in Amharic and English (Section 6.4).

4.2 Existing solutions and their limits

Approach	What it does well	Limitation for this problem
Browser reputation lists (Google Safe Browsing)	Very large coverage of reported malware and phishing; hash-prefix lookups protect privacy; real-time checks reduce the delay for unknown URLs [6][7].	Depends on a URL being known or quickly classified; global rather than Ethiopia-focused; the user is not shown which indicators were found.
Community feeds (PhishTank)	Free, frequently updated list of verified phishing URLs, useful as ground truth and for evaluation.	Reactive by design: listed only after reports are verified; URLs often go offline within hours.
Email and SMS filtering	Stops many lures before the user sees them.	Operates outside the browser; does not help when a link arrives through a chat or social web client. Much Ethiopian fraud arrives by SMS and calls [4][5].
User awareness campaigns (INSA, Ethio telecom)	Teach never to share one-time codes and to verify requests [3][4].	Cannot scale to every message; users still decide under pressure.
PhishGuard (this project)	Heuristic, explainable, local; checks URL, page, links and local-language cues even for domains never reported before.	Rule-based; detects little phishing aimed at unfamiliar brands from the URL alone; browser-only (Sections 7 and 9).

Row one summarizes public descriptions of Safe Browsing; the comparison is of design approaches, not a measured benchmark of the products against each other.

4.3 Security implications and design consequences

- **False positives are costly.** A noisy tool trains users to ignore warnings. PhishGuard therefore never blocks on a single rule: every signal contributes a weight to a score, and only a high score produces a blocking warning.

- **Explainability builds trust.** The popup lists every signal that fired, so the user can judge the verdict and so each result can be reviewed in the evaluation.
- **Privacy matters.** Page HTML can contain sensitive content. The engine runs on the user's own machine and accepts only local connections.
- **The tool itself is an attack surface.** Strings from untrusted pages are rendered with `textContent` rather than HTML, and the server validates input size and URL scheme.

5. System Architecture

PhishGuard has two parts: a Chrome extension (Manifest V3, JavaScript) that observes pages and presents verdicts, and a detection engine written in Python that scores them. The engine is a set of independent, testable modules feeding a single weighted scorer, which keeps each module easy to build, test and explain.

The extension cannot run Python directly, so the engine runs as a small local server (`server.py`, Python standard library only) bound to `127.0.0.1:8765`. For each page the content script sends the URL and page HTML to `POST /analyze`; for links the user hovers over, it sends the URL only. The server returns a JSON verdict containing the score, the verdict and the list of signals. This is the "client plus lightweight backend" option, with the backend on the user's own machine rather than a remote server, so no browsing data leaves the device.

Figure 1 shows the data flow. Module responsibilities are summarized below.

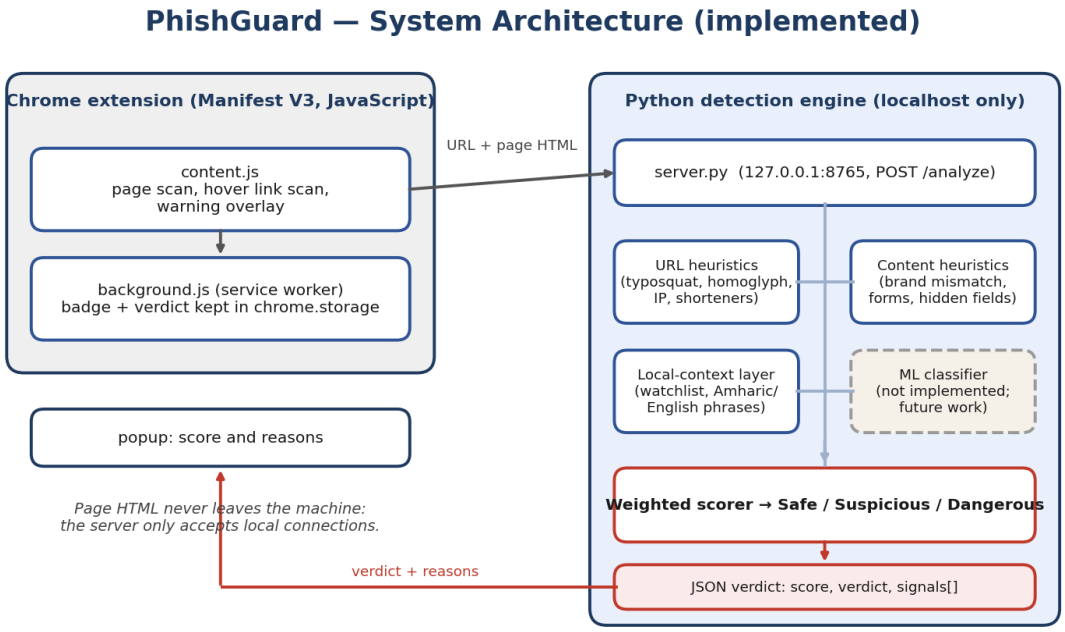


Figure 1 — PhishGuard system architecture (as implemented)

URL heuristics	Content heuristics	Local-context layer	ML classifier
Structure and domain checks; also used for pre-click link scanning.	DOM checks: brand/domain mismatch, forms, hidden fields.	Institution watchlist; Amharic and English scam phrases.	Stretch goal. Not implemented; see Section 12.

6. Implementation

The code is organized as `engine/` (the detection logic), `server.py` (local API), `extension/` (Chrome shell), `evaluate.py` (measurement) and `tests/`. All signals are returned as records with a name, a weight, a human-readable explanation and a source layer.

6.1 Scoring model

Every signal adds its weight to a score capped at 100. The score maps to a verdict: **Safe** below 20, **Suspicious** from 20 to 59, and **Dangerous** at 60 or above. The Suspicious threshold of 20 was chosen from the measured trade-off between detection and false positives (Section 7.3). Two rules refine this:

- **High-confidence floor.** A brand/domain mismatch on a page that also asks for a password raises the score to at least 80, because that combination is the strongest signal of a cloned login page.
- **Trusted domains.** If the host belongs to an official watchlist domain, almost all signals are suppressed so legitimate sites are not flagged. Two checks stay active: a password field over plain HTTP, and a login form that posts to a different site; the latter adds a further penalty because it can indicate a compromised page.

6.2 URL heuristics

The URL is parsed into scheme, host and path; the registered domain is derived with a small suffix table that includes Ethiopian suffixes (`com.et`, `gov.et`, `org.et`, `edu.et`, `net.et`). Signals and weights:

Signal	Weight	Meaning
`homoglyph`	60	Domain label equals a watchlist brand after normalizing look-alike characters.
`typosquat`	45	Within 1 edit (2 for labels of 8+ characters) of a watchlist brand.
`brand_wrong_suffix`	35	Same label as a brand, but not on its official domain.
`brand_in_domain`	30	Brand keyword appears as a hostname token on an unrelated domain.
`ip_host`	25	Raw IP address instead of a domain name.
`free_hosting_subdomain`	15	Subdomain of a free hosting or site-builder platform (for example pages.dev, netlify.app).
`risky_tld`, `gibberish_domain`	10 each	Domain ending frequently used for abuse (for example .top, .icu, .xyz); random-looking domain name.

Signal	Weight	Meaning
`long_digit_run`, `brand_other_cctld`	8 each	Four or more digits in a row in the hostname; a big international brand on a country domain (weak, since this is often legitimate).
`at_in_url`, `mixed_script`	15 each	`@` in the address; a domain label mixing Latin letters with another alphabet (a look-alike trick). Plain non-Latin domains such as .рф sites are not flagged.
`brand_in_path`	12	Brand keyword in the path of an unrelated domain.
`suspicious_words_host`	5 each, max 12	Words such as login, verify, secure, account in the hostname.
`no_https`, `url_shortener`, `many_subdomains`	10 each	Plain HTTP; known shortener; 4 or more dots in the host.
`many_hyphens`, `digit_heavy`, `long_url`, `suspicious_words_path`	8 / 6 / 5 / 4	Further weak indicators.

Only the single strongest brand-related signal is reported per URL, so that one lookalike is not counted several times. The last group of signals does not depend on any brand: they were added after the first evaluation showed that most real phishing URLs imitate brands outside the watchlist. Each is weak on its own, so a risky ending alone never produces a warning; several together do. A local institution's own domain name and hyphenated names such as m-pesa are matched inside hostnames.

6.3 Content heuristics

The page HTML is parsed with Python's standard `html.parser`, so the engine has no third-party dependencies and malformed HTML cannot crash it. Signals:

Signal	Weight	Meaning
`brand_mismatch`	40 (18 without password field)	The title, logo alt text or site name presents a watchlist brand, but the page is not hosted on that brand's domains. Body text is deliberately ignored so a news article that mentions a bank is not flagged.
`external_form_action`	25 (8 if no credentials)	A form submits to a different registered domain.
`password_over_http`	20	Password field on a page not served over HTTPS.
`offscreen_fields`	15	Text or credential inputs hidden or placed off-screen. Standard hidden inputs such as CSRF tokens are ignored.
`brand_mention_with_login`	25	A local institution is named two or more times on a page that asks for a password but is hosted elsewhere (catches clones whose title does not name the brand).
`link_text_mismatch`	15 (20 for 3 or more)	Link text shows one web address but the link goes to a different domain, a classic lure.
`login_on_risky_url`	15	Password page on an address that already shows warning signs (brand-independent; added by the scorer).

Signal	Weight	Meaning
`login_form`	5	Page asks for a password (context for the floor rule).

6.4 Local-context layer

The watchlist (`engine/data/watchlist.json`) holds the official domains and name variants of commonly impersonated institutions: Ethiopian banks, telebirr and M-PESA, Ethiopian Airlines, ERCA and INSA, plus widely impersonated international brands (PayPal, Google, Microsoft, Facebook, Instagram, Telegram, Apple, Amazon). Each entry records whether it has been verified against an official source. The list is intentionally small and curated rather than scraped.

The phrase matcher looks for scam-style wording in Amharic and English in the page title and visible text. The list holds 19 Amharic and 21 English phrases, written for this project from common scam themes: account blocked or about to close, urgency, prizes and cash rewards, and bonus offers such as free airtime or internet. Phrases are grouped by what they signal. Pressure phrases (threats, urgency, prizes, bonuses) are what scams rely on, so they drive the score; wording about passwords or verification is common on legitimate pages, so it only adds weight when a pressure phrase is also present. One pressure phrase adds 10 points, two or more add 20, and credential or verification wording alongside them adds 5. This grouping was added so that a legitimate Amharic login page, which naturally says ‘password’ and ‘verify’, is not flagged. Amharic has spelling variants (for example two spellings of the word for ‘free’), so variants need separate entries, and matching is on exact text.

The Amharic phrases were reviewed by the project author and judged correct. They were written from common scam themes rather than collected from real scam messages, so the list is a starting point; collecting real messages (anonymized) is listed under future work. The official domain of each of the nine local institutions was checked against its website on 26 October 2026 and marked as verified in the watchlist file.

6.5 Pre-click link scanning

The content script listens for the pointer resting on an external link for 300 ms. It sends only that link’s URL to the engine, which applies the URL heuristics, and shows a small tooltip when the verdict is Suspicious or Dangerous. Results are cached per link. Same-site links are skipped. This works on any web page the extension can access, including webmail and the web versions of social platforms and messengers, and directly targets the moment before the click. It cannot run inside native mobile apps or SMS.

6.6 Extension shell

- **Manifest V3** with a content script on http(s) pages, a background service worker, and a popup.
- **Service worker state.** Because the browser may terminate the service worker at any time, verdicts are saved in `chrome.storage` and are not kept only in memory. The toolbar badge shows ‘?’ for Suspicious, ‘!!’ for Dangerous, and ‘off’ when the local engine is not running.

- **Warnings.** Suspicious pages show a non-blocking banner naming the main concern. Dangerous pages show a full-page warning with the score and top reasons, with the options ‘take me to safety’ or ‘continue anyway’.
- **Popup.** Shows the verdict, the score, and every signal with its weight and source layer, which makes each decision reviewable.

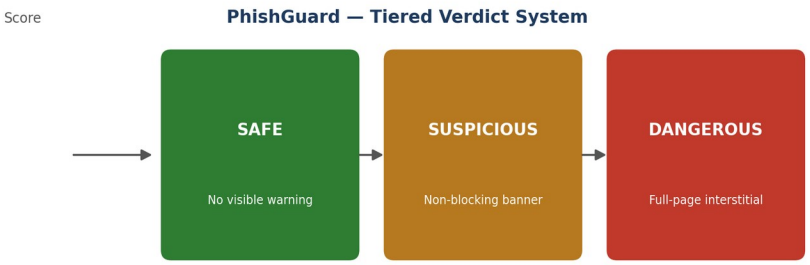


Figure 2 — Tiered verdict system: Safe, Suspicious, Dangerous

6.7 Local server and input handling

`server.py` uses only the standard library, listens on `127.0.0.1`, accepts `POST /analyze` with a JSON body, rejects bodies over 2 MB and URLs that are not http(s), and returns JSON. The content script truncates page HTML to 300 KB before sending.

7. Testing and Evaluation

7.1 Unit, functional and browser tests

A suite of 43 automated tests (`python -m unittest`) runs against the engine. All 43 pass. They cover:

Area	Tests	What is verified
URL heuristics	6	Official domains stay Safe; a homoglyph domain is Dangerous; a typosquat and an IP-address login URL are flagged; a shortener alone is not escalated; an ordinary site is Safe.
Content heuristics	7	A cloned login page is Dangerous; a news article mentioning a brand is not; the real brand’s own login page is Safe; an official domain posting to another site is flagged; CSRF hidden fields are ignored; off-screen fields are flagged; malformed HTML does not crash the engine.
Local context	7	Amharic scam wording is detected; clean text produces no phrase signal; password or verification wording alone is not flagged; prize and telecom-bonus wording is detected; adding credential wording to pressure phrases raises the score.
Helpers	2	Edit distance and registered-domain extraction (including `com.et`).
Generic URL features	6	Free-hosting subdomains, risky TLD alone is not enough, random-looking domains, stacked weak signals are flagged, normal large sites stay Safe.

Area	Tests	What is verified
Regression tests from the first evaluation	3	A big brand on a country domain (`google.fr`) and a brand-owned TLD (`.microsoft`) are no longer false positives; a local brand on the wrong suffix is still flagged.
Page-level features (link text, login on risky URL, brand named on a login page)	6	Each new rule fires on the intended case, and does not fire on matching link text, a plain login on a clean address, or a news page that merely names a local bank.
Internationalized domains	3	A plain Cyrillic domain and the false positive found in the seed 2 sample are no longer flagged; a Latin and Cyrillic mixed look-alike of a brand is flagged.
Local-brand rules	3	A local brand's own name inside a hostname is flagged; the hyphenated keyword `m-pesa` is matched; ordinary sites with common words are not.

The extension was loaded unpacked in Chrome and tested together with the running local engine. On a real Microsoft documentation page the popup showed Safe (0/100) and identified the official domain, and each demonstration page in Section 8 produced the expected result.

7.2 Evaluation method

Effectiveness was measured with three fixed sets, all run on 26 October 2026 with a fixed random seed:

- **Phishing set:** 500 URLs sampled at random from the PhishTank verified-online feed (`verified_online.csv`) [8]. Downloaded on 26 October 2026 at about 09:00 (local time).
- **Benign set:** 500 domains sampled at random from the Tranco top-1-million list [9]. Downloaded on 26 October 2026 at about 10:52 (local time); the list ID was not recorded.
- **Local-brand set:** 117 look-alike URLs of the nine local watchlist institutions (typos, look-alike characters, brand names inside other domains, wrong suffixes, 'bonus' and 'claim' style hostnames) plus the 9 official sites as the benign side. The set is generated by `eval/make_local_set.py`.

A URL counts as 'flagged' if its score reaches the Suspicious level (20) or higher. Reported measures are the detection rate (flagged phishing divided by all phishing), the false-positive rate (flagged benign divided by all benign) and precision. Two random samples (seeds 1 and 2) were drawn from the same files. Seed 1 was used while developing the rules. Seed 2 was drawn afterwards and was not used to choose the threshold or to add features; however, its two false positives were examined and one rule was corrected, so it is no longer a fully independent test and its figures should be read with that in mind. The main evaluation is URL-only, because live phishing pages cannot be safely fetched at scale and often go offline within hours; page-content signals are tested on locally hosted clone pages (Section 8).

Important limit of the local-brand set: its URLs are generated using the same kinds of tricks the detector looks for. It therefore measures whether the local-context layer covers the design

cases, not real-world accuracy. A set of real reported Ethiopian phishing URLs would be a stronger test and is listed under future work.

7.3 Results

How the results changed. The first run exposed weaknesses, which were fixed and re-measured:

Version of the engine	Sample	Detection	False pos.	Precision
Initial rules, Suspicious at 30	seed 1	0.2%	0.4%	33.3%
+ generic URL features and fixes, Suspicious at 30	seed 1	2.8%	0.0%	100%
same	seed 2	3.6%	0.2%	94.7%
Final, Suspicious at 20	seed 1	**12.6%**	**0.0%**	100%
Final, Suspicious at 20	seed 2	**14.2%**	**0.2%**	98.6%

Choice of threshold. The Suspicious threshold was chosen from the measured trade-off rather than guessed. Raising detection by flagging lower scores costs false positives:

Flag at score ≥	Seed 1 detection	Seed 1 false pos.	Seed 2 detection	Seed 2 false pos.
10	49.0%	5.6%	54.2%	6.2%
15	25.0%	0.0%	28.2%	0.8%
20 (chosen)	**12.6%**	**0.0%**	**14.2%**	**0.2%**
25	6.4%	0.0%	10.0%	0.0%
30	2.8%	0.0%	3.6%	0.0%
40	0.0%	0.0%	0.2%	0.0%
50 and 60	0.0%	0.0%	0.0%	0.0%

Twenty keeps false positives at or below 0.2% on both samples while detecting about one in seven phishing URLs. Below 20 the false-positive rate rises sharply (up to 5.6% and 6.2% at a threshold of 10), which would train users to ignore warnings. Dangerous (blocking) warnings stay at a score of 60.

Local-brand set. This is the test aimed at the project's main contribution:

Condition	Detected	Detection	False pos. (9 official)
Local-context layer OFF	9 of 117	7.7%	0
Local-context layer ON, first run	106 of 117	90.6%	0
Local-context layer ON, after fixing the 11 misses	117 of 117	**100%**	0

The 11 misses in the first run were a real gap: a hostname such as `awashbank-login-secure.com` was not matched because only the short keyword ‘awash’ was checked, and the hyphenated keyword `m-pesa` was never matched. The rules were fixed to also match a local institution’s own domain name and hyphenated names, and regression tests were added. Because those fixes were made after seeing which generated URLs failed, the 100% shows that the design now covers these cases; it should not be read as an independent accuracy figure.

What the results mean. On general phishing from PhishTank, URL-only scanning detects a small share (12.6% and 14.2%). Most of these URLs imitate brands that are not on the watchlist, use brand-new random-looking domains, or are hosted on legitimate platforms, so there is little in the address alone to inspect. The tool is much stronger where it was designed to be: impersonation of the local institutions on its watchlist, where the local-context layer raises detection from 7.7% to 100% on the generated set. The false-positive rate stays very low on popular sites (0.0% and 0.2%). These results are consistent with the design: the URL layer is a fast first filter, and the page-content layer (brand mismatch, external form posts, hidden fields) is what catches clones of unfamiliar brands once the page loads.

7.4 Example walkthroughs

Caught: look-alike login. `https://paypa1.com/login` produces a homoglyph signal (digit 1 for letter l, matching a watchlist brand, +60) and a suspicious path word (+4): 64, Dangerous.

Caught after the fix: local brand inside another domain. `https://awashbank-login-secure.com/verify` scored only 14 in the first local run (Safe) because the bank’s name was not matched inside the hostname. After the fix it is flagged.

Missed: unfamiliar brand on a risky domain. `https://br-atualiza-pendente.digital/` scores 10 (only the risky `.digital` ending): Safe at the chosen threshold. Nothing in the address names a watchlist brand. `https://sites.google.com/view/...` from the seed 2 sample scores 0 because the phishing page is hosted on a legitimate Google platform. Only the page content can reveal such cases.

False positives found and fixed. In the first evaluation, `https://google.fr/` scored 35 (brand on the wrong suffix) and `https://usgovcloud-core.microsoft/` scored 30 (brand name in the hostname). Big international brands legitimately use country domains, and `.microsoft` is the brand’s own domain ending. Both rules were corrected and regression tests added. In the first seed 2 run, two popular domains were flagged. One was an internationalized Russian domain (`.рф`): the encoded form of a non-Latin name always looks random, so the random-name and punycode rules misfired. That was a real bug; the rules now decode the name first and only flag a label that mixes Latin letters with another alphabet, and the re-run shows one false positive left out of 500. The remaining one, a short random-looking name on a `.cc` domain (`wxnxxqjh.cc`), scores exactly 20 from two weak signals together. It is a fair judgment call and shows the cost of the Suspicious threshold of 20, so it was left as it is.

8. Technical Demonstration

The demonstration used locally hosted training pages (in `demo/`), not live third-party phishing sites. The local 'collector' page discards form data. All results below match what was expected.

Page	Content	Observed result
Official Microsoft documentation page	Real legitimate site	Safe, 0/100; official domain recognized
Clone of a bank login (`cbe-birr-login.test`)	Bank brand in the title and logo text, password field, form posting to another domain	Dangerous, 100/100; full-page warning
Amharic and English prize and urgency page	Scam wording only	Suspicious; banner naming the scam-style wording (8 phrase matches)
Customer notice page (`link_mismatch.html`)	A local bank named repeatedly, a password form, and link text showing the bank's address while linking elsewhere	Dangerous, 75/100; full-page warning with the new signals listed in the popup
Message page with look-alike links	Six links, hover only	Tooltip on the bad links (for example Dangerous 64/100); none on the legitimate links

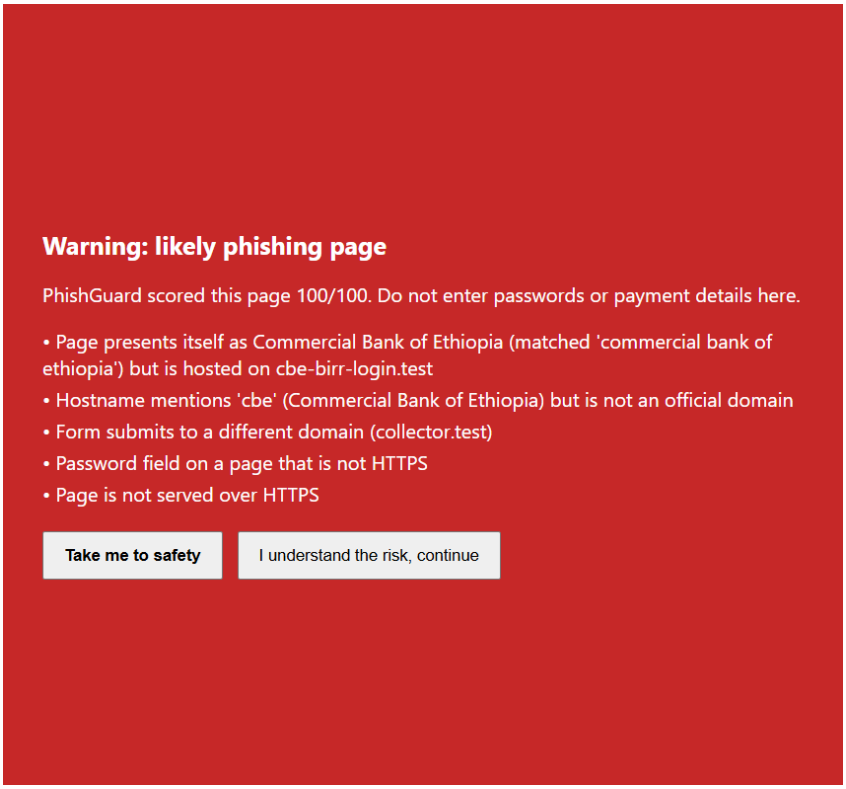


Figure 4 — Full-page warning on the cloned bank login: score 100/100 with the reasons

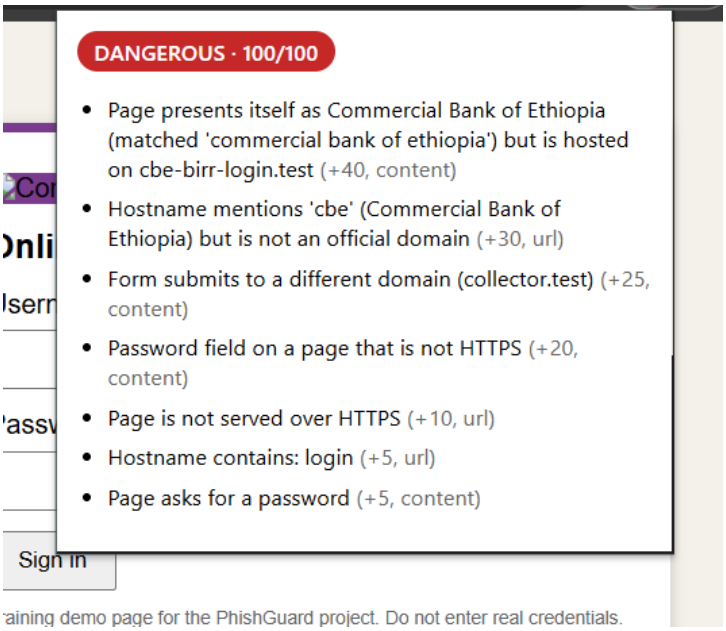


Figure 5 — Popup listing each signal with its weight and source layer

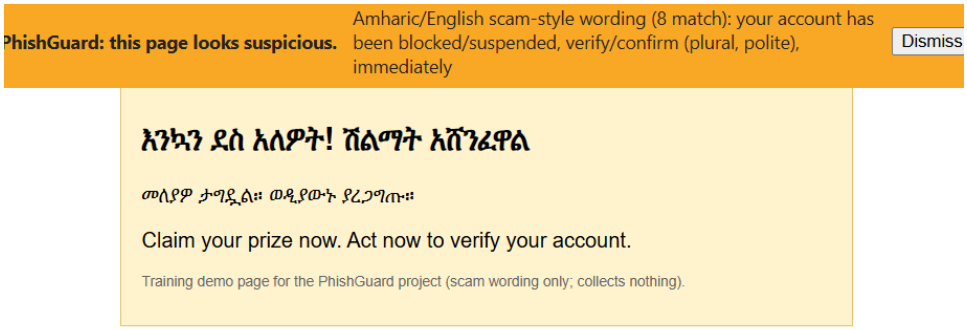


Figure 6 — Suspicious banner on the Amharic and English scam page

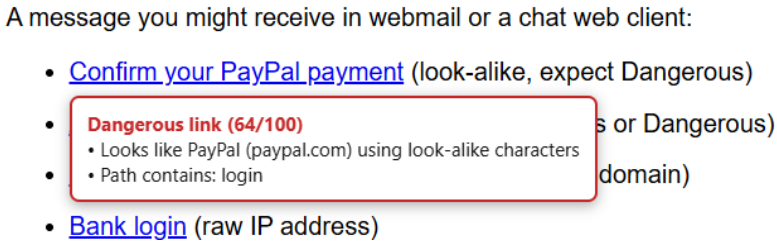


Figure 7 — Pre-click tooltip on a look-alike link (Dangerous, 64/100)

- Confirm your PayPal payment (look-alike, expect Dangerous)
- Awash Bank statement (typo, expect Suspicious or Dangerous)
- CBE Birr account update (brand inside another domain)
- Bank login (raw IP address)
- Real PayPal sign in (official, expect no tooltip)
- Wikipedia: Phishing (expect no tooltip)

Figure 8 — The same message page: no tooltip appears on the legitimate links

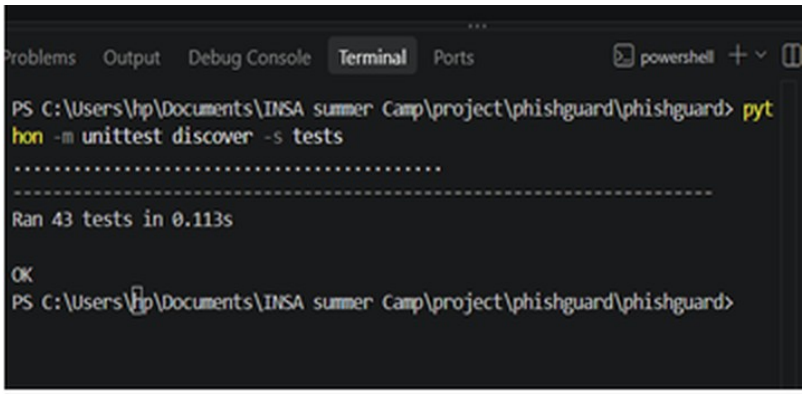


Figure 9 — Automated tests run in VS Code: 43 tests, all passing

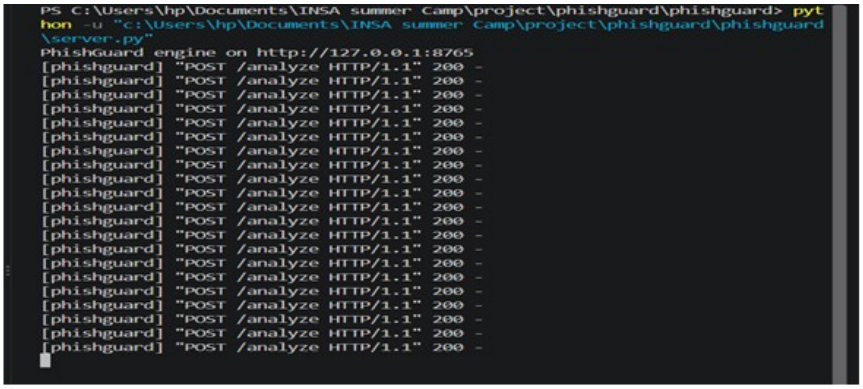


Figure 10 — Engine log during the demonstration: each page or link scan from the extension is answered with HTTP 200

9. Scope and Limitations

- **Low detection of general phishing from the URL alone.** On random PhishTank URLs the final detection rate is 12.6% to 14.2% (Section 7.3). Most phishing in that feed imitates brands outside the watchlist or uses new random-looking domains and legitimate hosting platforms.
- **Strength is local-brand impersonation.** The 100% result on the local-brand set is on generated URLs, not real reported attacks, and the rules were corrected after the first run on that same set.

- **Page-content layer not measured at scale.** It is verified by unit tests and the local demonstration pages, but not on a large sample of real phishing pages or of real benign pages (so the false-positive rate of the page-level rules on real sites is unknown), because live phishing pages cannot be safely fetched in bulk and go offline quickly.
- **Browser and platform.** Chrome (Manifest V3) only. The engine must be running locally; if it is stopped, the extension shows ‘off’ and does not protect.
- **Channels.** Pre-click scanning works only on web pages the extension can access, including webmail and the web versions of social platforms. It cannot see native mobile apps, SMS, voice calls or USSD, which are important channels for fraud in Ethiopia [4] [5].
- **Heuristic detection.** Rule-based with hand-set weights; no trained model. Attackers who study the rules can evade them. The weights were set from limited data.
- **Warning timing.** The Dangerous warning is an overlay shown after the page loads, not a block before it renders.
- **HTTPS check.** Extensions cannot read a site’s TLS certificate in Manifest V3, so the check only looks at whether the address uses HTTPS.
- **Shorteners and redirects.** URL-only scanning cannot see where a shortened or QR link leads.
- **Local lists.** The watchlist and Amharic phrase list cover a representative sample, are not exhaustive and depend on manual verification. Trusted-domain handling assumes official domains are not compromised.
- **Evaluation limits.** Results depend on the date and composition of the samples; live phishing URLs change daily.
- **Ethics.** Testing used publicly listed samples and locally hosted pages in a controlled environment, not live third-party systems.

10. Risks and Mitigations

Risk	Mitigation / outcome
Manifest V3 service workers can be stopped by the browser, losing in-memory state.	Verdicts are stored in `chrome.storage`; the content script re-requests analysis on each page load.
Live phishing URLs go offline within hours, making repeatable testing hard.	Fixed, dated samples plus local clone pages for repeatable testing.
Over-aggressive rules cause false positives and erode trust.	Weighted scoring instead of single-rule blocking; trusted-domain handling; thresholds tuned against the benign set.
The local engine is a new attack surface.	Localhost-only binding, input size and scheme validation, no HTML injection in the extension UI.
Domain-age lookups depend on rate-limited free APIs.	Left out of the core design; listed as future work.
Machine learning layer may not be finished.	Descoped; the rule-based layers form a complete, testable deliverable.

11. Development Timeline and Status

Figure 3 shows the original plan. The table records what was completed.

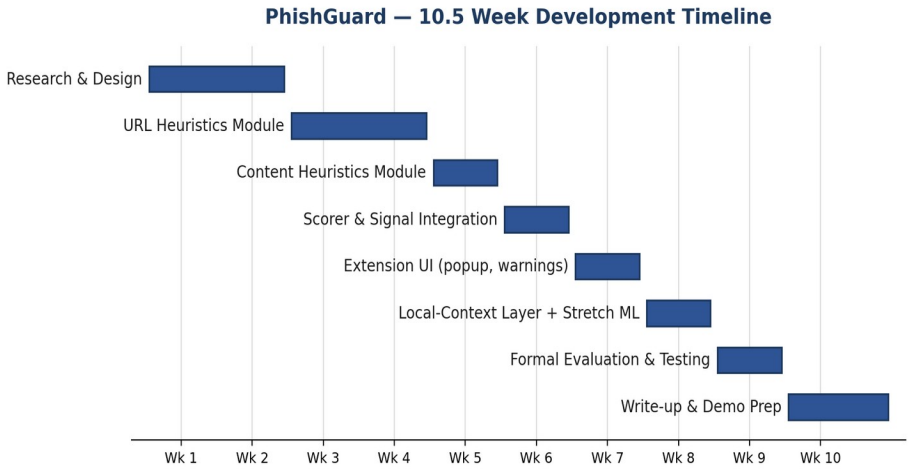


Figure 3 — Development timeline by week (original plan)

Phase	Status	Notes
Research and design	Done	Requirements, architecture, existing-solution analysis.
URL heuristics	Done	Implemented and unit-tested.
Content heuristics	Done	Implemented and unit-tested.
Scorer and integration	Done	Three-tier verdict with floor rule and trusted-domain handling.
Extension UI	Done	Popup, badge, banner, warning overlay, hover tooltip. Tested in Chrome.
Local-context layer	Done (starter lists)	Lists need verification and expansion.
ML classifier (stretch)	Not done	Descoped; see Future Work.
Formal evaluation	Done	PhishTank/Tranco (two seeds) and local-brand set run; rules improved after the first run. See Section 7.
Write-up and demo	Done	This report; demonstration run in Chrome with locally hosted pages (Section 8).

12. Future Work

- Verify and expand the Ethiopian watchlist and Amharic phrase list through a structured collection effort.
- Add and evaluate a lightweight machine-learning classifier on a public phishing URL dataset to complement the rules; this is the most direct way to raise detection of phishing aimed at brands outside the watchlist.

- Evaluate the page-content layer on a sandboxed collection of real phishing pages, and test the local layer on real reported Ethiopian phishing URLs rather than generated ones.
- Follow shortened links and redirects to the final destination before scoring.
- Block navigation before render using declarative network rules, and package the engine so no separate server process is needed.
- Support Firefox and other Chromium-based browsers.
- Explore SMS and messaging-app scam detection, since many local attempts arrive as text messages containing a malicious link or a USSD code.

13. Conclusion

Phishing is a persistent, human-centered attack, and reputation-based protection leaves gaps for newly created pages and for region-specific threats. PhishGuard is a working, explainable detection tool that combines URL analysis, page-content analysis, pre-click link scanning and a local-context layer for Ethiopian institutions and Amharic scam wording. It is implemented in Python behind a Chrome extension and covered by 43 passing automated tests; the demonstration in Chrome produced the expected result on every page.

The measured results are mixed and are reported as they are. On 500 random PhishTank URLs against 500 popular sites, URL-only detection reached 12.6% and 14.2% on two samples, with false positives at 0.0% and 0.2%; most real phishing in the feed imitates brands outside the watchlist, which a URL check cannot see. On look-alikes of the local institutions the tool is much stronger: the local-context layer raised detection from 7.7% to 100% of the generated test set. The first evaluation also exposed rule errors that were corrected, which is the purpose of testing. The project is clear about what it does not cover: SMS, voice and app-based fraud, and unfamiliar brands, which need a trained model and page-level analysis at scale.

References

[1] APWG, “APWG Year in Review: Scams Explode in 2025 via Social Media, SMS, Email and QR Codes,” press release, 23 February 2026.

[2] APWG, Phishing Activity Trends Report, 1st Quarter 2026, published 21 May 2026. https://docs.apwg.org/reports/apwg_trends_report_q1_2026.pdf

[3] “Ethiopia: INSA Warns of Widespread WhatsApp Cyberattacks Exploiting Seventh General Election,” allAfrica, February 2026. https://allafrica.com/stories/202602270614.html

[4] Ethio telecom, “Fraud Awareness.” https://ethiotelecom.et/?p=466216

[5] “Rising threats from digital scams” (interview with J. Somers, NetGuardians), Capital Ethiopia, December 2024. https://capitalethiopia.com/2024/12/09/rising-threats-from-digital-scams/

[6] Google Security Blog, “Better password protections in Chrome,” December 2019. https://security.googleblog.com/2019/12/better-password-protections-in-chrome.html

[7] TechCrunch, “Google’s Safe Browsing protection in Chrome goes real-time,” 14 March 2024. https://techcrunch.com/2024/03/14/googles-safe-browsing-protection-in-chrome-goes-real-time/

[8] PhishTank, verified phishing URL feed (data.phishtank.com), downloaded 26 October 2026 at about 09:00 (local time), used as the phishing sample.

[9] Tranco top-sites ranking (tranco-list.eu), downloaded 26 October 2026 at about 10:52 (local time), used as the benign sample. V. Le Pochat et al., “Tranco: A Research-Oriented Top Sites Ranking Hardened Against Manipulation,” NDSS 2019.